

# CV Intelligence Matcher using RAG + LLM

## Executive Summary

This project presents an AI-powered CV Intelligence Matcher system based on Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs). The system allows users to upload raw PDF resumes and ask natural language questions regarding candidate skills, experience, and qualifications.

Unlike traditional keyword-based recruitment systems, the proposed architecture performs semantic retrieval using vector embeddings and context-aware reasoning using a local LLM. The system extracts text from resumes, splits documents into semantic chunks, generates embeddings using HuggingFace models, stores vectors inside Qdrant, retrieves relevant chunks using similarity search, and injects the retrieved context into the LLM prompt to generate grounded responses.

The backend was implemented using FastAPI and containerized using Docker for scalable deployment.

The final system supports:

- Semantic resume search
- Natural language querying
- Arabic and English support
- Offline local LLM inference
- REST API architecture
- Dockerized deployment

## System Architecture

User -> FastAPI Backend -> PDF Processing -> PyMuPDF Text Extraction -> Chunking  
->HuggingFace Embeddings -> Qdrant Vector Database  
-> Semantic Retrieval -> Ollama + Mistral LLM -> Final Response

## Example API Responses:

### Upload Response

```
{ "message": "uploaded successfully",  
  "file": "resume.pdf" }
```

## Answer Response:

```
{ "query": "Who has Docker experience?",  
  "answer": "Khaled Yehia Abdallah Mohamed has Docker and Kubernetes skills." }
```

## API Documentation

| Endpoint                          | Description                  |
|-----------------------------------|------------------------------|
| <code>/upload</code>              | Upload PDF resumes           |
| <code>/process/{file_name}</code> | Process and vectorize PDF    |
| <code>/search</code>              | Semantic similarity search   |
| <code>/answer</code>              | RAG-based question answering |

## Embedding Model & Chunking Strategy

The project uses the HuggingFace embedding model: all-MiniLM-L6-v2

This model was selected because it is lightweight, fast, and optimized for semantic similarity tasks while generating high-quality 384-dimensional embeddings.

The extracted text is divided using:

`RecursiveCharacterTextSplitter`

with:

```
chunk_size = 1200  
chunk_overlap = 100
```

Chunking improves semantic retrieval accuracy by splitting long resumes into manageable sections. The overlap preserves contextual continuity and reduces information loss between chunks.

## Retrieval & RAG Logic

The querying pipeline follows the standard RAG workflow:

User Query -> Query Embedding -> Vector Similarity Search -> Top-K Retrieval -> Context Injection -> LLM Generation -> Final Response

The system retrieves the top semantic matches from Qdrant using cosine similarity. Weak matches are filtered using similarity thresholds to reduce hallucination and improve answer precision.

Strict prompting rules were applied:

- Do NOT guess
- Do NOT invent information
- Answer ONLY from the provided context

## **Docker Deployment Instructions :**

### **Build Containers**

docker compose build

### **Run the System**

docker compose up

### **Access URLs**

<http://localhost:8000/ui>

<http://localhost:8000/docs>

## **Conclusion**

The CV Intelligence Matcher demonstrates how Retrieval-Augmented Generation can combine semantic search, vector databases, and local LLMs to build intelligent document analysis systems. The project integrates NLP, embeddings, semantic retrieval, FastAPI APIs, and Dockerized deployment to provide accurate and context-aware resume analysis.